# L'Architecture Client-Serveur et le Protocole HTTP : Fonctionnement, Structure des Messages et Codes de Statut

**Thème principal :** Communication Web, Architecture Client-Serveur, Protocole HTTP, Méthodes GET/POST et Codes de Statut

**Synthèse exécutive :** Ce cours pose les fondements fondamentaux du fonctionnement du Web en analysant l'architecture Client-Serveur et le protocole HTTP (HyperText Transfer Protocol). Il explicite le fonctionnement asymétrique des échanges Web, la nature 'stateless' (sans état) de HTTP et ses mécanismes de contournement (cookies/sessions), la structure textuelle normalisée des messages HTTP (requêtes et réponses), la différenciation stricte entre les méthodes GET et POST sur le plan fonctionnel et sécuritaire, ainsi que le décodage systématique des codes de statut (2xx, 3xx, 4xx, 5xx) indispensables au débogage d'applications web.

## 1. L'Architecture Client-Serveur et la Nature 'Stateless' de HTTP

L'architecture client-serveur régit l'intégralité des échanges sur le réseau Web. Contrairement aux réseaux Pair-à-Pair (P2P) où chaque machine est à la fois cliente et serveuse, le Web repose sur un modèle strictement asymétrique. Le client (un navigateur web tel que Chrome, Firefox ou Safari) émet une requête HTTP à destination d'un serveur web (processus applicatif distant comme Apache, Nginx ou IIS). Le serveur écoute en permanence sur un port réseau standardisé, traite la demande (accès au système de fichiers, exécution de scripts backend, requêtes à une base de données) et retourne un message de réponse HTTP contenant la ressource ou une erreur. Le protocole HTTP est dit 'stateless' (sans état) : une fois la réponse transmise, la connexion est fermée ou réinitialisée, et le serveur oublie instantanément le client. Cette absence de mémoire garantit une excellente scalabilité (les serveurs n'ont pas à maintenir en mémoire vive des millions d'états de connexion active), mais interdit le fonctionnement natif d'applications complexes comme des boutiques e-commerce ou des espaces membres. Pour pallier cette contrainte, on utilise des cookies de session : le serveur transmet un identifiant de session dans un en-tête 'Set-Cookie', que le navigateur enregistre et renvoie automatiquement dans les en-têtes de toutes les requêtes ultérieures adressées au même domaine.

- Asymétrie Client-Serveur : Rôles stricts entre le client actif qui prend l'initiative et le serveur passif qui écoute sur un port réseau (80 pour HTTP, 443 pour HTTPS).
- Protocole Stateless (Sans état) : Caractéristique fondamentale où chaque transaction requête-réponse est totalement indépendante et amnésique. Le serveur ne conserve aucune mémoire des requêtes passées.
- Gestion de Session et Cookies : Mécanisme d'en-tête permettant d'injecter un identifiant unique (Session ID) transmis à chaque requête pour simuler un état persistant (panier d'achat, espace connecté).

***Cas concret enseignant :*** *Le professeur compare le protocole HTTP à un guichetier amnésique : chaque fois que vous vous présentez devant lui, même si c'était il y a deux secondes, il vous traite comme un inconnu total. Pour qu'il se souvienne de vous, vous devez lui montrer votre badge (le cookie de session) à chaque phrase. De plus, lors d'une simple visite sur la page d'accueil de l'IUT de Bayonne, le navigateur n'envoie pas une seule requête unique, mais des dizaines de requêtes HTTP simultanées pour récupérer séparément le fichier HTML de base, puis chaque feuille de style CSS, script JavaScript et image associée.*

## 2. Structure des Messages HTTP et Comparatif des Méthodes (GET vs POST)

Un message HTTP transite sous forme textuelle brute. Une requête comporte : 1. La ligne de requête (verbe HTTP, URI de la ressource, version du protocole) ; 2. Les en-têtes (paires clé-valeur fournissant le contexte) ; 3. Une ligne vide servant de séparateur strict (CRLF) ; 4. Le corps (Body) contenant les données. La réponse du serveur adopte la même structure mais commence par une ligne de statut (ex: 'HTTP/1.1 200 OK'). Le choix entre les verbes GET et POST est un enjeu de conception et de sécurité crucial : - GET sert à lire des données sans modifier le serveur

(méthode dite sûre et idempotente). Ses paramètres sont accolés à l'URL sous forme de chaîne de requête (?champ=valeur). En raison de cette visibilité, GET ne doit JAMAIS transporter de données sensibles. - POST est utilisé pour soumettre des données altérant le système (inscription, validation de formulaire). Les paramètres transitent dans le corps (Body) de la requête, permettant de transférer de grands volumes d'informations (fichiers, images) et évitant l'affichage des variables dans la barre d'adresse.

- Format Textuel Normalisé : Un message HTTP (requête ou réponse) est structuré en texte clair avec une première ligne descriptive, des en-têtes (Headers), une ligne vide obligatoire (CRLF) et un corps optionnel (Body).
- En-têtes Critiques : 'Host' (obligatoire en HTTP/1.1 pour le Virtual Hosting), 'User-Agent' (identification du navigateur/OS), 'Content-Type' (type MIME des données).
- Méthode GET : Méthode de lecture/consultation, sûre et idempotente, transmettant ses variables dans l'URL (Query String). Seconde limite de taille (~2048 caractères) et vulnérable aux fuites de données.
- Méthode POST : Méthode de création/modification d'état, encapsulant ses variables dans le corps (Body) de la requête, sans limite de taille et masquée dans la barre d'adresse.

***Cas concret enseignant :*** *Le professeur met en garde contre l'utilisation absurde de GET pour transmettre des identifiants : soumettre un mot de passe en GET l'affiche en clair dans l'URL de l'écran, le sauvegarde dans l'historique du navigateur et l'enregistre en clair dans les fichiers de logs du serveur web. C'est la vulnérabilité parfaite au 'shoulder surfing' (regard par-dessus l'épaule).*

## 3. Décodage et Classification des Codes de Statut HTTP

Chaque réponse émise par le serveur inclut un code numérique à trois chiffres permettant au navigateur ou à l'application cliente d'interpréter instantanément l'issue de la demande : - 2xx (Succès) : La requête a été reçue, comprise et acceptée (ex: 200 OK, 201 Created). - 3xx (Redirection) : Indique que le client doit émettre une nouvelle requête vers un autre emplacement (ex: 301 Moved Permanently, 304 Not Modified exploitant le cache). - 4xx (Erreur Client) : La requête contient une anomalie. Exemples : 401 Unauthorized (authentification requise), 403 Forbidden (accès refusé par droits), 404 Not Found (ressource absente sur le disque du serveur). - 5xx (Erreur Serveur) : La demande du client est valide mais le serveur a rencontré une défaillance interne lors du traitement. Exemples : 500 Internal Server Error (crash de script PHP/Node.js, erreur SQL), 503 Service Unavailable (serveur saturé ou en maintenance).

- Ligne de Statut de Réponse : Première ligne d'une réponse HTTP (ex: HTTP/1.1 200 OK) indiquant le résultat du traitement à l'aide d'un code numérique à 3 chiffres.
- Classification des Familles : 1xx (Information), 2xx (Succès), 3xx (Redirection), 4xx (Erreur Client), 5xx (Erreur Serveur).
- Frontière de Responsabilité 4xx vs 5xx : 4xx indique une requête mal formulée ou ciblant une ressource invalide (faute du client) ; 5xx indique un crash du code backend ou de la base de données alors que la requête était valide (faute du serveur).

***Cas concret enseignant :*** *Le professeur raconte une anecdote personnelle : à ses débuts, une erreur de syntaxe dans son script PHP (un simple point-virgule manquant lors d'une connexion à la base de données) faisait planter le serveur lors de la validation d'un formulaire, renvoyant systématiquement une erreur 500 Internal Server Error. Les utilisateurs paniquaient en pensant que le site était piraté alors qu'il s'agissait simplement d'une faute de frappe dans son code backend.*

## ■ Questions & Interventions des Étudiants en Classe

**Question étudiant :** Monsieur, quelle est la différence concrète entre GET et POST pour envoyer un formulaire de connexion ?
**Réponse du professeur :** Avec GET, les identifiants et le mot de passe sont collés directement dans l'URL (ex: login.php?user=matei&pass;=1234), ce qui les rend visibles dans l'historique, les logs serveurs et sur l'écran. Avec POST, les identifiants sont encapsulés dans le corps (Body) du message HTTP : ils n'apparaissent ni dans l'URL ni dans l'historique, ce qui est obligatoire pour des raisons de confidentialité élémentaire (bien qu'il faille toujours utiliser HTTPS pour les chiffrer sur le réseau).

## ■ Exercice & Démonstration d'Application

**Énoncé :** Analysons la requête HTTP brute suivante. Identifier la méthode utilisée, la ressource demandée, l'hôte cible, le rôle de l'agent utilisateur, et expliquer le comportement exact du serveur avec son code de statut si la ressource sollicitée est absente : GET /images/logo.png HTTP/1.1 Host: iutbayonne.univ-pau.fr User-Agent: Mozilla/5.0

**Résolution pas à pas :**
1. Ligne de requête ('GET /images/logo.png HTTP/1.1') : - Méthode HTTP : GET (demande de lecture/récupération d'une ressource sans modification de l'état du serveur). - URI demandée : '/images/logo.png' (chemin d'accès absolu au fichier sur le serveur web). - Protocole : HTTP version 1.1. 2. En-tête Host ('Host: iutbayonne.univ-pau.fr') : - Spécifie le nom de domaine du serveur destinataire. Cet en-tête est rendu obligatoire en HTTP/1.1 pour permettre l'hébergement virtuel ('Virtual Hosting'), permettant à un unique serveur physique/IP d'héberger plusieurs sites web distincts. 3. En-tête User-Agent ('User-Agent: Mozilla/5.0') : - Transmet au serveur la signature du logiciel client (type de navigateur et système d'exploitation). 4. Scénario en cas d'absence de la ressource : - Si le fichier 'logo.png' n'existe pas dans le dossier '/images/' du serveur web, la demande cible un emplacement invalide. - L'erreur provenant de la saisie/demande d'une mauvaise URL par le client, la responsabilité incombe au client. - Le serveur renverra une réponse HTTP de la famille 4xx, précisément le code '404 Not Found'.

**Résultat : Méthode : GET | Ressource sollicitée : /images/logo.png | Hôte cible : iutbayonne.univ-pau.fr | Rôle User-Agent : Identification du client | Code en cas d'absence : 404 Not Found (Erreur sous la responsabilité du client).**

## ■ Auto-Évaluation Flash d'Examen

**Q : Pourquoi qualifie-t-on le protocole HTTP de 'stateless' (sans état) et quelle est sa conséquence sur la gestion des sessions ?**

*R : HTTP est dit 'stateless' car le serveur ne conserve aucune mémoire des requêtes précédentes d'un même client. Chaque cycle requête-réponse est isolé. La conséquence est qu'il est impossible de maintenir nativement une session utilisateur (panier d'achat, espace connecté) sans ajouter des mécanismes complémentaires comme des cookies de session (transmettant un Session ID à chaque requête).*

**Q : Quelle est la différence fondamentale de responsabilité entre une erreur HTTP de la série 4xx et une erreur de la série 5xx ?**

*R : Une erreur 4xx (ex: 404 Not Found) indique que la responsabilité incombe au client (mauvaise URL, manque d'autorisation, ressource inexistante). Une erreur 5xx (ex: 500 Internal Server Error) indique que la requête du client était valide, mais que le serveur a subi une défaillance interne (crash d'un script backend, base de données inaccessible).*

**Q : Pourquoi la méthode HTTP GET ne doit-elle jamais être employée pour transmettre des données sensibles comme un mot de passe ?**

*R : La méthode GET transmet ses paramètres directement dans l'URL sous forme de Query String. Les données sensibles apparaissent ainsi en clair dans la barre d'adresse de l'écran (exposition au shoulder surfing), sont sauvegardées dans l'historique du navigateur et sont enregistrées en clair dans les journaux de logs des serveurs web et proxys.*